

UNIVERSIDAD DE GUADALAJARA

ARQUITECTURA DE SISTEMAS DE INTELIGENCIA ARTIFICIAL

CUGDL

Prerrequisitos
ASIA:
Machine Learning II

Mtro. David Echavarry

10 de marzo de 2026

Agenda

- Objetivo de clase
 - Predicción del modelo (Model Prediction)
 - Error de predicción (Prediction Error)
 - Función de pérdida (Loss Function)
 - Error de entrenamiento vs error de prueba
 - Generalización (Generalization)
 - Overfitting y Underfitting
 - Bias–Variance Tradeoff
 - Métricas de evaluación (Evaluation Metrics)
 - Métricas de Clasificación
 - Métricas de Regresión
 - Selección de modelo (Model Selection)
 - Cross-Validation
 - k-fold Cross Validation
 - Pipeline conceptual de Machine Learning

Objetivo

Idea clave

- Introducir los principios generales del aprendizaje automático relacionados con la **predicción, el error, la evaluación y la selección de modelos.**

Predicción del modelo (Model Prediction)

Predicción del modelo

- En aprendizaje supervisado, el objetivo es **aprender una función** que relacione las entradas con las salidas.
- Existe una función verdadera desconocida que genera los datos:

$$y = f(x)$$

- El modelo aprende una **función aproximada** llamada **hipótesis**:

$$\hat{y} = h(x)$$

- La salida $\hat{y}$ se denomina **predicción del modelo**, mientras que y representa el **valor real**.
- El objetivo del aprendizaje automático es que la predicción del modelo **se aproxime lo mejor posible al valor verdadero**.

Predicción del modelo (Model Prediction)

Ejemplo:

- Supongamos que queremos predecir el **precio de una casa** a partir de su tamaño.
- Entrada del modelo:

$$x = \text{tamaño de la casa (m}^2\text{)}$$

- Valor real observado:

$$y = \text{precio real de la casa}$$

- El modelo aprende una función $h(x)$ que produce una predicción:

$$\hat{y} = h(x)$$

- Por ejemplo, si una casa tiene $x = 120 \text{ m}^2$, el modelo puede predecir:

$$\hat{y} = 250,000$$

- Esta cantidad representa la **predicción del modelo**.

Predicción vs valor real

Comparación entre predicción y realidad

- Una vez que el modelo genera una predicción $\hat{y}$, podemos compararla con el **valor real** observado y .
- Esta comparación permite evaluar **qué tan cerca está la predicción del modelo de la realidad**.
- En la práctica, es común que:

$$\hat{y} \neq y$$

- Por ejemplo:
 - Valor real del precio de la casa: $y = 260,000$
 - Predicción del modelo: $\hat{y} = 250,000$
- La diferencia entre estos valores nos permite medir **qué tan correcto o incorrecto es el modelo**.

Error de predicción (Prediction Error)

Error de predicción

- Una vez que el modelo genera una predicción $\hat{y}$, es posible compararla con el **valor real** observado y .
- La diferencia entre estos valores se denomina **error de predicción**.

$$error_i = y_i - \hat{y}_i$$

- donde:
 - y_i representa el **valor real** o **ground truth**.
 - $\hat{y}_i$ representa la **predicción del modelo**.
- El error de predicción mide **qué tan lejos está la predicción del modelo del valor real**.

Error de predicción (Prediction Error)

Ejemplo

- Supongamos que un modelo predice el **precio de una casa**.
- Valor real observado:

$$y = 260,000$$

- Predicción del modelo:

$$\hat{y} = 250,000$$

- El error de predicción se calcula como:

$$error = y - \hat{y}$$

$$error = 260,000 - 250,000 = 10,000$$

- Esto indica que la predicción del modelo se desvió del valor real en **10,000 unidades**.

Función de pérdida (Loss Function)

Función de pérdida

- El error de predicción mide la diferencia entre el valor real y y la predicción del modelo $\hat{y}$.
- Para evaluar el desempeño del modelo de forma sistemática, se utiliza una **función de pérdida**.

$$L(y, \hat{y})$$

- La función de pérdida cuantifica **qué tan mala es una predicción realizada por el modelo**.
- Durante el entrenamiento, el objetivo del aprendizaje automático es **encontrar la hipótesis h que minimice la pérdida total**.

$$\min_{h \in H} \sum_{i=1}^N L(y_i, \hat{y}_i)$$

- Es decir, buscamos el modelo que produzca **la menor pérdida posible sobre los datos**.

Función de pérdida (Loss function)

Ejemplo

- Supongamos que queremos predecir el **precio de una casa**.
- Valor real observado:

$$y = 260,000$$

- Predicción del modelo:

$$\hat{y} = 250,000$$

- Si utilizamos la función de pérdida de **error cuadrático**:

$$L(y, \hat{y}) = (y - \hat{y})^2$$

- Entonces la pérdida es:

$$L = (260000 - 250000)^2$$

$$L = (10000)^2$$

$$L = 100000000$$

- Esta cantidad representa **cuánto se equivocó el modelo en esta predicción**.

Error de entrenamiento vs error de prueba

Error de entrenamiento

- El **error de entrenamiento** se calcula utilizando los datos del **training set**.
- Este error mide qué tan bien el modelo se ajusta a los datos utilizados durante el aprendizaje.

$$E_{train} = \frac{1}{N} \sum_{i=1}^N L(y_i, \hat{y}_i)$$

- Un error de entrenamiento bajo indica que el modelo ha aprendido los patrones presentes en los datos de entrenamiento.

Error de entrenamiento vs error de prueba

Error de prueba

- El **error de prueba** se calcula utilizando el **test set**, que contiene datos no vistos durante el entrenamiento.
- Este error mide la capacidad del modelo para **generalizar a datos nuevos**.

$$E_{test} = \frac{1}{M} \sum_{i=1}^M L(y_i, \hat{y}_i)$$

- En aprendizaje automático, el objetivo es obtener un modelo con **buen desempeño en datos nuevos**.

Generalización (Generalization)

Generalización

- El objetivo del aprendizaje automático no es solamente ajustarse a los datos de entrenamiento, sino **generalizar a datos nuevos**.
- Un modelo **generaliza bien** cuando es capaz de producir predicciones correctas en datos que **no fueron utilizados durante el entrenamiento**.
- En otras palabras, el modelo debe aprender **patrones generales del problema**, no simplemente memorizar los ejemplos observados.
- Idealmente, buscamos que:

$$E_{train} \approx E_{test}$$

- Cuando el error de prueba es bajo, podemos decir que el modelo tiene **buena capacidad de generalización**.

Generalización (Generalization)

Ejemplo

- Supongamos que entrenamos un modelo para predecir el **precio de casas** usando datos de entrenamiento.
- Después del entrenamiento, calculamos el error en el conjunto de entrenamiento:

$$E_{train} = 12,000$$

- Luego evaluamos el modelo en el conjunto de prueba (datos no vistos):

$$E_{test} = 13,500$$

- Observamos que:

$$E_{train} \approx E_{test}$$

- Esto indica que el modelo produce errores similares en datos conocidos y en datos nuevos, por lo que podemos decir que el modelo **generaliza bien**.

Overfitting y Underfitting

Underfitting

- Ocurre cuando el modelo es **demasiado simple** para capturar los patrones presentes en los datos.
- Como resultado, el modelo presenta **errores altos tanto en entrenamiento como en prueba**.

$$E_{train} \text{ alto, } E_{test} \text{ alto}$$

- En este caso, el modelo **no aprende adecuadamente la relación entre las variables**.

Overfitting y Underfitting

Overfitting

- Ocurre cuando el modelo es **demasiado complejo** y se ajusta excesivamente a los datos de entrenamiento.
- El modelo aprende incluso el **ruido presente en los datos**.

$$E_{train} \text{ bajo, } E_{test} \text{ alto}$$

- En este caso, el modelo **memoriza los datos de entrenamiento** y no generaliza bien.

Underfitting y Overfitting

Ejemplo de Underfitting

- Supongamos que queremos construir un modelo para **clasificar correos electrónicos** como *spam* o *no spam*.
- El modelo utiliza una regla demasiado simple, por ejemplo:
 - Si el correo contiene la palabra “free”, entonces es spam.
- Sin embargo, muchos correos spam no contienen esa palabra y algunos correos normales sí la contienen.
- Como resultado, el modelo comete muchos errores tanto en entrenamiento como en prueba.

$$E_{train} \text{ alto, } E_{test} \text{ alto}$$

- El modelo es **demasiado simple para capturar el patrón real del problema**.

Underfitting y Overfitting

Ejemplo de Overfitting

- Ahora supongamos que entrenamos un modelo extremadamente complejo con un conjunto pequeño de correos.
- El modelo aprende reglas muy específicas para cada correo del conjunto de entrenamiento.
- Por ejemplo, podría aprender reglas como:
 - Si el correo contiene exactamente ciertas combinaciones de palabras, entonces es spam.
- Estas reglas funcionan perfectamente en el entrenamiento, pero fallan en correos nuevos.

E_{train} bajo, E_{test} alto

- En este caso, el modelo **memoriza los ejemplos en lugar de aprender patrones generales**.

Bias–Variance Tradeoff

Bias y Variance

- El error de un modelo puede analizarse en términos de dos componentes principales:
 - **Bias**: error debido a suposiciones demasiado simples del modelo.
 - **Variance**: error debido a alta sensibilidad a los datos de entrenamiento.
- Un modelo con **alto bias** suele ser demasiado simple y produce **underfitting**.
- Un modelo con **alta variance** suele ser demasiado complejo y produce **overfitting**.
- En aprendizaje automático buscamos un equilibrio entre ambos errores.

$$\text{Error total} = \text{Bias}^2 + \text{Variance} + \text{Noise}$$

Métricas de evaluación (Evaluation Metrics)

Evaluación del modelo

- Una vez entrenado un modelo, es necesario medir **qué tan bien realiza sus predicciones**.
- Para ello utilizamos **métricas de evaluación**, que resumen el desempeño del modelo sobre un conjunto de datos.
- Estas métricas se calculan a partir de las **predicciones del modelo** y los **valores reales**.

Predicciones ($\hat{y}$) vs Valores reales (y)

- Las métricas permiten:
 - comparar diferentes modelos
 - evaluar la capacidad de generalización
 - seleccionar el mejor modelo para un problema
- Las métricas utilizadas dependen del tipo de problema:
 - **Clasificación**
 - **Regresión**
- Las métricas pueden usarse para evaluar cualquier modelo del mismo tipo de problema.
- **Cada métrica mide un aspecto diferente del error.**

Métricas para Clasificación

Métricas de evaluación en Clasificación

- En problemas de **clasificación**, el objetivo del modelo es asignar una etiqueta o clase a cada observación.
- Para evaluar el desempeño de estos modelos se utilizan diversas métricas que comparan las **predicciones del modelo** con las **etiquetas reales**.
- Las métricas más utilizadas en clasificación incluyen:
 - **Accuracy**
 - **Precision**
 - **Recall** (o Sensitivity)
 - **F1-score**
 - **Confusion Matrix**
- Estas métricas permiten analizar el comportamiento del modelo desde diferentes perspectivas.

Métricas para Clasificación

Confusion Matrix

- La **confusion matrix** es una tabla que permite analizar el desempeño de un modelo de clasificación.
- Compara las **predicciones del modelo** con las **etiquetas reales**.
- En un problema de clasificación binaria se definen cuatro posibles resultados:
 - **True Positive (TP)**: el modelo predice correctamente la clase positiva.
 - **True Negative (TN)**: el modelo predice correctamente la clase negativa.
 - **False Positive (FP)**: el modelo predice positivo cuando en realidad es negativo.
 - **False Negative (FN)**: el modelo predice negativo cuando en realidad es positivo.
- A partir de estos valores se definen la mayoría de las **métricas de evaluación en clasificación**.

Métricas para Clasificación

Ejemplo de Confusion Matrix

Ejemplo

- Supongamos que entrenamos un modelo para clasificar correos electrónicos como spam o no spam.
- Evaluamos el modelo sobre un conjunto de prueba y obtenemos una **confusion matrix** de la siguiente forma:

Etiquetas Reales	Predicción del modelo	
	Spam	Ne Spam
	True Positive (TP): 50	FN = 10
	False Positive (FP): 5	TN = 85

- En este ejemplo obtenemos:
 - TP = 50, TN = 85
 - FP = 5, FN = 10

Métricas para Clasificación

Accuracy

- **Accuracy** mide la proporción de predicciones correctas realizadas por el modelo.
- Es decir, indica qué porcentaje de las predicciones totales fueron correctas.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

- Donde:
 - *TP*: True Positives
 - *TN*: True Negatives
 - *FP*: False Positives
 - *FN*: False Negatives
- Esta métrica es fácil de interpretar, pero puede ser engañosa cuando las clases están desbalanceadas.
- Esto introduce perfectamente la motivación para las siguientes métricas:
 - Precision
 - Recall
 - F1-Score

Métricas para Clasificación

Ejemplo de limitaciones de Accuracy

- Supongamos que queremos construir un modelo para detectar **correos spam**.
- Tenemos un conjunto de datos con **1000 correos electrónicos**:
 - 950 correos **no spam**
 - 50 correos **spam**
- Ahora imaginemos un modelo muy simple que **clasifica todos los correos como no spam**.
- En este caso el modelo acertaría:

950 predicciones correctas de 1000

$$Accuracy = \frac{950}{1000} = 95 \%$$

- Aunque la **accuracy es muy alta (95 %)**, el modelo **no detecta ningún correo spam**.
- Esto ocurre porque las clases están **desbalanceadas**.
- Por lo tanto, **accuracy puede ser engañosa cuando las clases están desbalanceadas**.

Métricas para Clasificación

Precision

- **Precision** mide qué proporción de las predicciones positivas realizadas por el modelo son realmente correctas.
- En otras palabras, responde a la pregunta:

*De todos los ejemplos que el modelo clasificó como positivos,
¿cuántos realmente pertenecen a la clase positiva?*

$$Precision = \frac{TP}{TP + FP}$$

- Donde:
 - *TP*: True Positives
 - *FP*: False Positives
- Una alta precision indica que el modelo **comete pocos falsos positivos**.

Métricas para Clasificación

Ejemplo de Precisión

- Supongamos que un modelo detecta correos **spam**.
- El modelo predice que **60 correos son spam**.
- De esos correos:
 - 50 realmente son spam ($TP = 50$)
 - 10 no son spam ($FP = 10$)
- Entonces la precision es:

$$Precision = \frac{50}{50 + 10}$$

$$Precision = 0,83$$

- Esto significa que el **83 % de los correos clasificados como spam realmente lo eran.**

Métricas para Clasificación

Recall

- **Recall** mide qué proporción de los ejemplos positivos reales fueron correctamente identificados por el modelo.
- En otras palabras, responde a la pregunta:

*De todos los ejemplos que realmente son positivos,
¿cuántos fueron detectados por el modelo?*

$$\text{Recall} = \frac{TP}{TP + FN}$$

- Donde:
 - *TP*: True Positives
 - *FN*: False Negatives
- Un alto recall indica que el modelo **detecta la mayoría de los casos positivos**.

Métricas para Clasificación

Ejemplo de Recall

- Supongamos que tenemos un sistema para detectar **correos spam**.
- En el conjunto de prueba existen **80 correos spam reales**.
- El modelo detecta correctamente:

$$TP = 60$$

- Pero deja pasar algunos correos spam:

$$FN = 20$$

- Entonces el recall es:

$$Recall = \frac{60}{60 + 20}$$

$$Recall = 0,75$$

- Esto significa que el modelo detecta **el 75 % de los correos spam**.

Métricas para Clasificación

F1-score

- En muchos problemas existe un equilibrio entre **Precision** y **Recall**.
- El **F1-score** combina ambas métricas en un solo valor.
- Se define como la **media armónica** entre precision y recall:

$$F1 = 2 \frac{Precision \cdot Recall}{Precision + Recall}$$

- Esta métrica es especialmente útil cuando:
 - las clases están **desbalanceadas**
 - queremos equilibrar **falsos positivos y falsos negativos**
- Un valor alto de F1-score indica que el modelo tiene **buena precision y buen recall**.

Métricas para Clasificación

Ejemplo de F1-score

- Supongamos que un modelo de detección de spam obtiene:

$$Precision = 0,83$$

$$Recall = 0,75$$

- Entonces el F1-score es:

$$F1 = 2 \frac{0,83 \times 0,75}{0,83 + 0,75}$$

$$F1 \approx 0,79$$

- Este valor resume el equilibrio entre la capacidad del modelo para:
 - detectar correctamente spam
 - evitar clasificar correos normales como spam

Métricas para Regresión

Evaluación en problemas de regresión

- En problemas de **regresión**, el modelo predice un **valor numérico continuo**.
- Para evaluar el desempeño del modelo, se mide la diferencia entre:

$$y \text{ (valor real)} \quad y \quad \hat{y} \text{ (predicción del modelo)}$$

- Esta diferencia se resume mediante diferentes **métricas de error**.
- Las métricas más utilizadas en regresión incluyen:
 - **Mean Absolute Error (MAE)**
 - **Mean Squared Error (MSE)**
 - **Root Mean Squared Error (RMSE)**
 - **R^2 (Coefficient of Determination)**
- Estas métricas permiten evaluar qué tan cerca están las predicciones del modelo de los valores reales.

Métricas para Regresión

Mean Absolute Error

- El **Mean Absolute Error (MAE)** mide el promedio de los errores absolutos entre las predicciones y los valores reales.

$$MAE = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

- Esta métrica indica, en promedio, **cuánto se equivoca el modelo en sus predicciones**.
- Una ventaja del MAE es que:
 - es fácil de interpretar
 - todos los errores tienen el mismo peso

Métricas para Regresión

Ejemplo (MAE)

- Supongamos que un modelo predice el **precio de casas**.
- Valores reales:

$$y = [200000, 250000, 300000]$$

- Predicciones del modelo:

$$\hat{y} = [210000, 240000, 310000]$$

- Calculamos los errores absolutos:

$$|y - \hat{y}| = [10000, 10000, 10000]$$

- Entonces:

$$MAE = \frac{10000 + 10000 + 10000}{3}$$

$$MAE = 10000$$

- En promedio, el modelo se equivoca **10,000 unidades** en sus predicciones.

Métricas para Regresión

Mean Squared Error (MSE)

- El **Mean Squared Error (MSE)** mide el promedio de los errores al cuadrado entre las predicciones del modelo y los valores reales.

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

- A diferencia del MAE, esta métrica **penaliza más los errores grandes**.
- Esto ocurre porque los errores se elevan al cuadrado.
- Por esta razón, el MSE es muy utilizado durante el **entrenamiento de modelos**.

Métricas para Regresión

Ejemplo de MSE

- Supongamos que tenemos los siguientes valores reales:

$$y = [200000, 250000, 300000]$$

- Y las predicciones del modelo:

$$\hat{y} = [210000, 240000, 310000]$$

- Calculamos los errores al cuadrado:

$$(y - \hat{y})^2 = [10000^2, 10000^2, 10000^2]$$

$$= [100000000, 100000000, 100000000]$$

- Entonces:

$$MSE = \frac{100000000 + 100000000 + 100000000}{3}$$

$$MSE = 100000000$$

Métricas para Regresión

Root Mean Squared Error (RMSE)

- El **Root Mean Squared Error (RMSE)** es la raíz cuadrada del MSE.

$$RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2}$$

- Esta métrica tiene la ventaja de que se expresa en **las mismas unidades que la variable objetivo**.
- Por esta razón, suele ser más **fácil de interpretar** que el MSE.

Métricas para Regresión

Ejemplo de RMSE

- En el ejemplo anterior obtuvimos:

$$MSE = 100000000$$

- Entonces el RMSE es:

$$RMSE = \sqrt{100000000}$$

$$RMSE = 10000$$

- Esto significa que el error promedio del modelo es de aproximadamente **10,000 unidades**.

Métricas para Regresión

Coefficient of Determination R^2

- El **Coefficient of Determination** (R^2) mide qué proporción de la variabilidad de los datos es explicada por el modelo.
- Se define como:

$$R^2 = 1 - \frac{\sum_{i=1}^N (y_i - \hat{y}_i)^2}{\sum_{i=1}^N (y_i - \bar{y})^2}$$

- Donde:
 - y_i : valor real
 - $\hat{y}_i$: predicción del modelo
 - $\bar{y}$: promedio de los valores reales
- El valor de R^2 generalmente se encuentra entre:

$$0 \leq R^2 \leq 1$$

- Un valor cercano a 1 indica que el modelo explica bien la variabilidad de los datos.

Métricas para Regresión

Ejemplo de R^2

- Supongamos que entrenamos un modelo para predecir el **precio de casas**.
- Después de evaluar el modelo obtenemos:

$$R^2 = 0,82$$

- Esto significa que el modelo explica aproximadamente el:

82 %

- de la variabilidad observada en los precios de las casas.
- El restante 18 % corresponde a factores que el modelo **no logra explicar**.

Resumen de métricas de clasificación

Interpretación de las métricas

- **Confusion Matrix:** permite visualizar cuántas predicciones fueron correctas y en qué tipos de errores se equivoca el modelo en una matriz.
- **Accuracy:** indica qué porcentaje de las predicciones del modelo fueron correctas.
- **Precision:** indica qué tan confiables son las predicciones positivas del modelo.
- **Recall:** indica qué tan bien el modelo logra detectar los casos positivos reales.
- **F1-score:** combina precision y recall en una sola métrica para evaluar el equilibrio entre ambas.

Resumen de métricas de Regresión

Interpretación de las métricas

- **MAE**: indica el error promedio absoluto entre las predicciones del modelo y los valores reales.
- **MSE**: mide el error promedio al cuadrado, penalizando más los errores grandes.
- **RMSE**: representa el error promedio del modelo en las mismas unidades que la variable predicha.
- R^2 : indica qué proporción de la variabilidad de los datos es explicada por el modelo.

Model Selection

Model Selection

- En aprendizaje automático es común tener **varios modelos posibles** para resolver un mismo problema.
- El objetivo de la **selección de modelos** es elegir el modelo que **generaliza mejor a datos nuevos**.
- Para ello, los modelos se comparan utilizando **métricas de evaluación**.
- El modelo seleccionado debe tener un buen desempeño en datos que **no fueron utilizados durante el entrenamiento**.
- Por esta razón se utilizan diferentes conjuntos de datos:
 - Training set
 - Validation set
 - Test set

Ejemplo de Model Selection

Ejemplo

- Supongamos que entrenamos tres modelos para predecir el precio de casas:
 - Modelo A
 - Modelo B
 - Modelo C
- Después de evaluarlos en el conjunto de validación obtenemos:
 - Modelo A: $RMSE = 12,000$
 - Modelo B: $RMSE = 9,500$
 - Modelo C: $RMSE = 11,200$
- El modelo con menor error es **Modelo B**.
- Por lo tanto, seleccionamos este modelo para evaluarlo finalmente en el **test set**.

Cross Validation

Cross Validation

- La **cross validation** es una técnica utilizada para evaluar el desempeño de un modelo de manera más robusta.
- En lugar de utilizar una sola división entre entrenamiento y validación, los datos se dividen en **varios subconjuntos**.
- El modelo se entrena y evalúa varias veces utilizando diferentes divisiones de los datos.
- Esto permite obtener una estimación más confiable del desempeño del modelo.
- Una de las técnicas más utilizadas es la **k-fold cross validation**.

k-fold Cross Validation

k-fold Cross Validation

- En **k-fold cross validation**, el conjunto de datos se divide en k partes iguales llamadas **folds**.
- El proceso consiste en:
 - entrenar el modelo con $k - 1$ folds
 - evaluar el modelo con el fold restante
- Este proceso se repite k veces, utilizando un fold diferente como conjunto de validación en cada iteración.
- Finalmente, el desempeño del modelo se calcula como el **promedio de los resultados obtenidos**.

Ejemplo de k-fold Cross Validation

Ejemplo

- Supongamos que tenemos un conjunto de datos con **1000 observaciones**.
- Si utilizamos **5-fold cross validation**, los datos se dividen en 5 partes de 200 observaciones cada una.
- El modelo se entrena y evalúa cinco veces:
 - Iteración 1: fold 1 como validación
 - Iteración 2: fold 2 como validación
 - Iteración 3: fold 3 como validación
 - Iteración 4: fold 4 como validación
 - Iteración 5: fold 5 como validación
- El desempeño final del modelo se obtiene promediando los resultados de las cinco evaluaciones.

Pipeline conceptual de Machine Learning

Proceso general en Machine Learning

- El desarrollo de un modelo de Machine Learning sigue una serie de pasos fundamentales:
 - 1 Definir el problema y los datos disponibles.
 - 2 Dividir los datos en **training set**, **validation set** y **test set**.
 - 3 Entrenar el modelo utilizando el **training set**.
 - 4 Evaluar el desempeño del modelo utilizando **métricas de evaluación**.
 - 5 Comparar diferentes modelos mediante **model selection**.
 - 6 Validar el modelo utilizando técnicas como **cross validation**.
 - 7 Evaluar finalmente el modelo con el **test set**.
- El objetivo final es obtener un modelo que **generalice correctamente a datos nuevos**.

Pipeline conceptual de Machine Learning

Flujo general del aprendizaje automático

Datos → Modelo → Predicciones → Error → Métricas → Selección de modelo → Validación

- Este flujo resume el proceso general para entrenar y evaluar modelos de Machine Learning.
- Los conceptos estudiados aplican **independientemente del algoritmo utilizado**.

Bibliografía

-  Russell, S., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.
-  Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.
-  Murphy, K. P. (2012). *Machine Learning: A Probabilistic Perspective*. MIT Press.